

Toward High-Assurance AI

Safety by Design for Autonomous Systems

Abdelhamid Bakhta

StarkWare

<https://github.com/AbdelStark>

April 2026

Abstract. AI safety is a broad umbrella spanning misuse, malfunctions, human and social harms, and systemic risks. We address one cross-cutting weakness visible across many of these settings: the structural distance between what developers and deployers of AI systems claim about behavior and what outside parties can independently verify. We call this the *integrity gap*. As AI systems move from advisory roles into physically embedded, agentic, and regulated deployments where failures are not recoverable, empirical evaluation, interpretability research, and process controls remain necessary but cease to be sufficient: such settings increasingly require stronger evidence for bounded safety-relevant claims under explicit assumptions. We argue for *safety by design*: a deployment-time evidence architecture that integrates formal safety specifications, verifiable computation, hardware attestation and provenance, privacy-preserving computation, and structured safety cases. Each primitive exists in partial form today; none is yet production-ready for frontier-scale deployment. We do not claim that assurance mechanisms resolve alignment or substitute for governance; we argue that they close an increasingly important and underdeveloped subset of AI safety. We make four contributions: we propose a layered architecture whose composed output is an *assurance bundle* that a third party can independently evaluate; name the *integrity gap* as the deployment-time evidence failure this architecture targets; introduce an *assurance claim ladder* that separates execution, bounded-property, and real-world-safety claims; and survey each layer’s maturity while framing a concrete research agenda for *high-assurance AI*.

1. Introduction

As AI systems move into settings where post-hoc correction is costly, delayed, or impossible, the central safety question changes.

Many deployments still fit the familiar cycle of evaluation, monitoring, and iterative improvement; those tools remain necessary. But autonomous agents can act over long horizons on delegated authority, physical AI systems translate inference into force, and regulated workflows may require external justification for particular decisions. In those settings, benchmark performance and internal logs are not enough. The question becomes: *can outside*

parties verify, independently and with high confidence, what a system actually did, under what conditions, and against which stated constraints?

Before proposing an answer, it helps to locate the question inside the wider field. AI safety is a broad umbrella. It includes deliberate misuse of AI systems, malfunctions and loss-of-control scenarios, human and social harms arising from large-scale deployment, and systemic risks at the level of institutions and markets (Russell et al., 2015; Slattery et al., 2024). We do not claim to address all of these. Our contribution sits in the *assurance, integrity, and accountability* dimensions of that umbrella: the cross-cutting deficiency that runs through many of the settings above is the structural difficulty outside parties have in verifying, independently of the provider, what an AI system actually did and whether it did what was claimed. We call this the **integrity gap**.¹

The term is not meant to rename every accountability problem in AI. Its job is narrower: to isolate the deployment-time evidence failure that remains after one accepts the need for evaluations, governance, safety cases, and alignment research. The contribution of this paper is therefore architectural rather than terminological. We ask what kind of evidence pipeline would let a third party connect four things that are usually separated: a stated safety property, the computation that was actually run, the runtime environment in which it ran, and a safety case that records the assumptions and residual risks.

Three questions become pressing once the gap is named. First, which safety-relevant claims can be made machine-checkable rather than merely asserted? Second, what technical mechanisms produce evidence in the right form—independently verifiable, without requiring discretionary provider cooperation at verification time once the relevant artifacts have been emitted? Third, how should those mechanisms compose with empirical alignment, interpretability, and institutional governance into a coherent assurance argument? This paper is a vision paper about infrastructure and composition: we do not present a new proof system or empirical result, but synthesize existing technical strands into a coherent high-assurance agenda. The sections that follow develop a systematic answer to all three.

Across all three, the honest answer is the same: not yet—not in a sufficiently general and independently verifiable form for high-assurance deployment. The necessary assurance infrastructure exists only in fragmented and uneven maturity. Several component primitives already provide partial assurance in bounded settings; a composed, deployment-ready stack does not yet exist. Empirical evaluation, interpretability research, process controls, and contractual trust remain the dominant mechanisms. They are valuable. They are not sufficient.

More precisely: the integrity gap is the distance between what high-assurance AI deployment requires—stronger evidence for bounded safety-relevant claims under explicit assumptions—and what the field currently produces.

We make four contributions. First, we propose a **composed assurance stack**, a lay-

¹Prior work has named adjacent concepts: the *accountability gap* in the legal-computational literature (Kroll et al., 2017; Raji et al., 2020), and more broadly the auditability and transparency problems in AI governance. We prefer *integrity gap* to emphasize that the obstacle in the settings we consider is architectural rather than procedural: current deployment structures make independent verification infeasible, not merely unexercised.

ered deployment-time evidence architecture whose joint output is an *assurance bundle*: a structured, independently evaluable artifact a third party can assess without access to the provider’s internal weights or operational logs. Second, we characterize the *integrity gap* precisely as the evidence failure this architecture targets: what it is, why it is structural rather than a temporary engineering limitation, and why it matters specifically for high-stakes deployment contexts. Third, we introduce an *assurance claim ladder* that separates three nested categories of claim (execution integrity, bounded-property satisfaction, and real-world safety) and argue that conflating them is a recurring category error in current AI safety discourse. Fourth, we survey each layer (formal verification, verifiable computation, attestation and provenance, privacy-preserving computation, and safety cases) with care to distinguish what each can prove today from what it cannot, and articulate a concrete research agenda for safety by design. Structured comparison to adjacent research clusters is deferred to §7, where it can be concrete rather than thematic.

We are not arguing that current AI safety work is wrong or misguided. RLHF, interpretability, red-teaming, and evaluation infrastructure are necessary components of responsible AI development. What they do not produce is machine-checkable, independently auditable, formally bounded evidence—and closing that gap is precisely what we argue for. Safety by design does not replace safety by evaluation. It adds the layer that evaluation alone cannot provide.

We use *high-assurance AI* throughout to denote AI systems deployed in settings where behavioral claims must be independently verifiable beyond what empirical evaluation alone can support: that is, systems whose deployment context demands evidence, at levels 1 and 2 of the assurance claim ladder introduced in §3, of execution integrity and bounded-property satisfaction, not only of distributional performance. The term is scoped to deployment conditions, not to any specific model class or capability level.

2. The Integrity Gap

2.1 What Assurance Means

The term *assurance* has a specific meaning in safety-critical engineering. In avionics, automotive systems, and nuclear power, assurance refers to the production of evidence, structured, documented, often formally verified, that a system satisfies its required properties under specified conditions. That evidence is typically organized into a **safety case**: a structured argument, supported by diverse artifacts, that links system behavior to stated guarantees (Clymer et al., 2024).

The critical property of assurance evidence, as understood in those fields, is that it is *independently evaluable*. An auditor who did not build the system can examine the evidence and form a defensible judgment about whether the safety argument holds. The evidence is not simply a developer’s claim that everything is fine.

This is the property that deployment-event-level AI safety practice largely lacks.

Current AI safety mechanisms produce important information about system behavior (empirical statistics, capability evaluations, alignment training histories, interpretability analyses). But little of this information is packaged as deployment-event-bound, machine-

checkable evidence. An auditor examining whether a frontier AI system behaved according to its stated policy during a particular deployment cannot, today, obtain a compact artifact that proves it did. They must rely on the provider’s logs, the provider’s claims, and whatever external testing they can conduct on a system they do not control.

For early-stage development, this is manageable. At scale, it becomes a structural problem.

2.2 Why the Gap Is Structural

The integrity gap is not a gap in effort or intent. It is a gap in *infrastructure*. The field has invested heavily in techniques for shaping and evaluating what AI systems do; it has invested much less in the complementary infrastructure for producing verifiable artifacts about what AI systems actually did.

Three scenarios make the structural nature of this concrete.

Model substitution. A cloud provider offers an AI API and represents that queries are answered by a specified model. A customer relies on this model for a regulated workflow. There is no technical mechanism by which the customer can verify that the representation is true. The provider could substitute a different model without the customer’s knowledge. This is not an allegation against any particular provider; it is a property of the current deployment architecture. Without verifiable computation, model identity claims are structurally unilateral.

Agentic delegation. A user delegates a long-horizon task to an autonomous agent. The agent takes hundreds of actions over several hours, some of which have external consequences (sending messages, executing payments, modifying files). The user needs to understand what happened and why. Current logging and monitoring provide a partial record, but not a record whose integrity is guaranteed by anything outside the systems of the party being audited.

Physical AI in a regulated context. An autonomous medical device makes a decision that contributes to a patient outcome. A regulator wants to audit the decision. What evidence exists? Without a deployment-time assurance architecture, the answer can collapse to whatever the manufacturer chose to record, in whatever format it chose, accessible only through its cooperation. The point is not that regulated medical AI lacks all oversight; it is that event-level evidence can still remain provider-mediated unless the deployment architecture is built to emit it.

In each case, a party outside the deployment chain needs to evaluate a claim about what happened, but the evidence is not in a form they can independently verify. That is what the integrity gap means in practice. Figure 1 shows the structural contrast between this condition and what an assurance stack would provide.

2.3 What Empirical Methods Cannot Do

Empirical safety methods (evaluations, red-teaming, RLHF, monitoring) provide probabilistic evidence about system behavior across a distribution of conditions. They are valuable

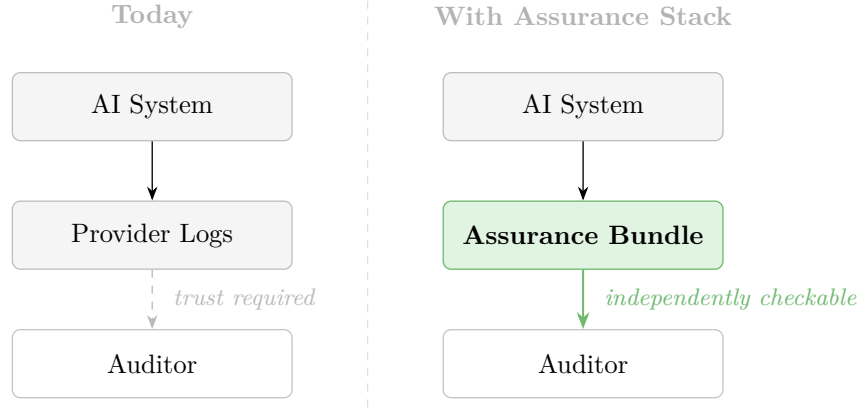


Figure 1: The integrity gap in structural terms. In the current deployment chain (left), an auditor examining AI behavior must rely on provider-controlled logs and unilateral claims; evaluating compliance requires trusting the party being evaluated. With an assurance stack (right), the system produces structured artifacts—some machine-checkable (cryptographic proofs, attestation reports), others independently evaluable by expert reviewers (safety cases)—that can be assessed without discretionary ex post provider cooperation once protocol-compliant artifacts have been emitted. The gap is not about provider honesty; it is about whether the architecture makes independent verification *possible*.

for characterizing average behavior, identifying failure modes, and improving systems over training iterations. They are the right tools for shaping what systems do in development.

Their limitation in high-assurance deployment is fundamental rather than incidental: they characterize behavioral distributions, not behavioral guarantees. A model that passes all evaluations can still behave in unanticipated ways on out-of-distribution inputs. A red-team that finds no critical failures cannot prove the system has none. An interpretability analysis that identifies the dominant features of a decision cannot, by itself, prove that a different decision pathway was unavailable.

This is not a criticism of empirical methods. It is a statement about the range of claims they can support. Even the most ambitious agenda for evaluating frontier systems for extreme-risk capabilities makes the same point explicitly: behavioral evaluations bound what a system is likely to do in tested conditions, not what it will do in deployment, and are a starting point for safety arguments rather than a substitute for them (Shevlane et al., 2023). High-stakes deployment settings (surgery, aviation, nuclear facility management, legal adjudication) have often required a stronger evidentiary pattern than sampled evaluation alone: bounded claims, explicit assumptions, and independently reviewable supporting evidence. Narrow, regulated AI applications in some of these sectors (medical-device software in regulated workflows, automotive ADAS under ISO 26262) already face structured evidence requirements—but those requirements do not yet reach the assurance level we propose and do not cover the architectural scope of frontier, general-purpose AI deployment. For that wider class, the integrity gap is the name for the current condition.

2.4 The Stakes of Inaction

The integrity gap is not a static condition. Two dynamics can make it worse over time if left unaddressed.

First, architectural defaults compound. Each generation of AI systems deployed without machine-checkable evidence risks normalizing that absence. Regulatory frameworks calibrate to what the industry actually produces; if providers never generate verifiable artifacts, there is little pressure surface on which to build requirements for them. The window for establishing assurance as a design requirement, rather than a costly retrofit, is easiest to use early and may become narrower as deployment architectures harden.

Second, the history of safety-critical engineering argues for early investment. Aviation did not add formal certification after a generation of unverifiable aircraft; it built certification infrastructure as the industry scaled. Pharmaceuticals did not introduce clinical trial methodology after accepting untested treatments as the normal basis for care. In both cases, the value of the safety infrastructure came from building it before failure cases accumulated at scale. AI may face a comparable choice. If assurance infrastructure is not built proactively, as a research and engineering discipline, it is more likely to be built reactively, under conditions that favor speed over correctness.

3. A Composed Assurance Stack

3.1 No Single Primitive Is Enough

The history of technical solutions to hard problems tends to generate single-mechanism proposals. Encryption will make AI outputs trustworthy. Interpretability will make AI behavior transparent. Zero-knowledge proofs will make AI execution verifiable. Each of these claims contains real insight and real limitation.

Encryption ensures confidentiality, not integrity. Interpretability can reveal what representations drove a decision, but not whether the execution was faithful to a specified policy. Zero-knowledge proofs can certify that a committed model executed correctly on a committed input, but they cannot certify that the specification was correct, that the model was aligned, or that the environmental inputs corresponded to reality.

Before describing the layers themselves, it is worth naming the distinction the rest of the argument depends on. Claims about AI systems come in three nested levels, and confusing them is a recurring category error in this area. We will call the distinction the *assurance claim ladder*.

- **Level 1: Execution integrity.** The claimed computation actually occurred: the advertised model was executed on the advertised input and produced the observed output. Verifiable computation and attestation bear directly on this level.
- **Level 2: Bounded-property satisfaction.** A formally specified behavioral predicate (an authority envelope, an output-region constraint, a formal fairness proxy) is satisfied under explicit assumptions. Formal verification and specification discipline bear on this level; level 1 evidence is a precondition.

- **Level 3: Safety in the world.** The deployed system, in its operational context, causes no harm the safety argument was supposed to exclude. No assurance stack can substitute for alignment research, human oversight, and institutional governance at this level; what it can do is make residual risk the explicit subject of the remaining argument, instead of the whole of it.

Each level depends on the one below it and is strictly stronger than it. We argue that the field currently lacks mature infrastructure for levels 1 and 2, and therefore has to treat claims about level 3 on trust. A structural limit divides levels 2 and 3 specifically: for rich open-world behavioral intents, a finite formal specification can be satisfied by a system that violates the underlying intent it was meant to capture—a form of Goodhart’s Law applied to behavioral contracts, which we formalize as the *specification-intent gap* in §4.1. The rest of the argument turns on that distinction, and we will refer back to the ladder whenever a claim is at risk of being read one rung too high.

The claim ladder identifies how strong an assurance claim is; the stack specifies which mechanisms can support claims at each rung. The two frameworks are complementary rather than redundant. Level-1 execution claims draw most directly on verifiable computation and attestation; level-2 bounded-property claims additionally depend on formal specification and verification; and level-3 deployment-safety claims still require a safety case that carries explicit assumptions and residual risk. Confusing *strength* with *coverage* is one reason single primitives are often asked to do more than they can.

No single mechanism addresses the full span of those claims. The right response is a composed architecture in which each mechanism addresses a specific layer, and the layers together support a bounded deployment-assurance argument under explicit assumptions.

One clarification on scope: the stack addresses the threat model of *unverifiable claims*, not adversarial inputs. It does not defend against an attacker manipulating model inputs to cause misclassification or induce unintended behavior. It defends against the structural condition in which no party outside the deployment chain can check whether the system did what was claimed. These are distinct problems. Conflating them leads to misplaced expectations about what any assurance layer can deliver.

The actors in this threat model are concrete: a developer or deployer that emits claims and artifacts, artifact-producing infrastructure (provers, attestors, provenance systems), a third-party verifier, and an institutional decision-maker that decides what evidence is sufficient. In scope are unilateral claims, model substitution, missing runtime binding, and inconsistencies across artifacts that an outside party can detect. Out of scope are malicious specification authors who choose bad properties in the first place, compromised hardware roots of trust, collusion among all artifact producers, and sensor or world-state attacks not captured by the provenance and control envelope. The stack narrows where trust must be placed; it does not eliminate trust.

3.2 The Stack

We propose five layers, shown in Figure 2. They are not sequential; they are compositional. (Throughout this section, *Layer* refers to a component of the assurance stack; *Level* refers to

a rung of the claim ladder introduced above—the two frameworks are orthogonal.) Layer 1 is foundational for bounded-property claims and for interpreting the bundle, but not every level-1 artifact depends on a behavioral specification: an attestation report or proof of inference can establish committed execution before anyone has decided whether the specified property is the right one. Layers 2–4 each produce a distinct class of artifact, and a given deployment may draw on all of them or on a subset appropriate to its threat model and regulatory context. Layer 5 is the integration layer in which the other layers’ artifacts become a structured, auditable argument.

Layer 1: Formal Safety Specifications and Verification. Before any proof can be produced, the property to be proved must be stated. Formal safety specifications translate behavioral requirements (“this system must not recommend actions that violate constraint X ,” or “this agent must not exceed its delegated authority”) into machine-checkable form. The same layer also includes the formal-verification step that checks bounded properties against those specifications under stated assumptions. It is foundational for bounded-property claims; the quality of that part of the assurance argument depends on the precision and completeness of the specifications.

This is currently one of the hardest layers. Specifying rich behavioral properties for open-world AI systems in formal languages remains largely unsolved. The difficulty of the problem does not reduce its necessity. An assurance argument built on informal behavioral descriptions is not an assurance argument; it is a characterization.

Layer 2: Verifiable Computation. Given a committed model and a committed input, can we prove that the observed output was produced by executing that model on that input? Verifiable computation, and specifically zero-knowledge proofs of neural network inference, provides a mechanism for this. It allows a party who did not witness the computation to verify, with cryptographic confidence, that a specific execution occurred. The intellectual precedent is *proof-carrying code* (Necula, 1997): the producer of a computation bears the burden of producing a compact proof, and the consumer performs cheap verification rather than extending unilateral trust. Applied to AI inference, the structural shift is the same—the burden of evidence moves from the auditing party (who must otherwise re-run or replicate the computation) to the provider (who generates the proof once, at inference time). The analogy has a structural limit: proof-carrying code certifies that compiled code satisfies its stated source-level specification, whereas ZK-of-inference certifies only that a specific weight tensor was executed on a specific input. Conformance to a behavioral specification—the question of whether the model *should* have been trusted to produce that output—remains a separate obligation discharged by Layer 1. The accountability-architecture insight transfers; the epistemic content does not.

This is the layer most directly relevant to model identity claims, execution audit trails, and compliance reporting. It answers the question: *did the thing it claims to have done actually happen?*

Layer 3: Attestation, Roots of Trust, and Provenance. Verifiable computation addresses the software execution. Attestation addresses the hardware and execution en-

vironment. A remote attestation report, backed by a hardware root of trust, provides evidence that the computation occurred on a specific, verified device running specific, measured firmware. Provenance standards provide evidence about the origin and transformation history of the inputs.

Together, these mechanisms support claims about the physical and infrastructural chain: not just that the model ran correctly, but that it ran on *this measured device*, over inputs with a recorded provenance chain, under a stated set of sensor and capture assumptions.

Layer 4: Privacy-Preserving Computation. Many high-stakes AI settings involve multiple stakeholders with conflicting confidentiality requirements: healthcare providers who cannot share patient data, financial institutions subject to data-residency rules, multi-agency government systems where inputs are classified. Privacy-preserving computation (primarily fully homomorphic encryption and secure multi-party computation) allows AI inference to occur over sensitive inputs without exposing them, while still supporting verifiable outputs.

This layer is not strictly required for every assurance argument, but it is essential for settings where privacy constraints and verifiability requirements pull in opposite directions. Without it, the choice between verifying the computation and protecting the inputs becomes a forced trade-off. With it, both requirements can potentially be met.

Layer 5: Safety Cases. The layers above generate artifacts: proof certificates, attestation reports, provenance manifests, privacy guarantees. A safety case is the structured argument that integrates those artifacts into a bounded deployment-assurance claim. Borrowed from safety-critical engineering practice, a safety case makes explicit what properties are being claimed, what evidence supports each claim, what assumptions the argument depends on, and what residual risks remain unaddressed.

Safety cases provide the integrative layer that turns a collection of technical artifacts into an auditable argument. They are the interface between the technical assurance stack and the institutional or regulatory actors who need to evaluate bounded assurance claims. They do not make level-3 safety machine-checkable; they make the judgment structure inspectable.

Adversary assumptions. Each layer carries its own adversary model, and the composed stack inherits their union. Layer 2 assumes a cryptographically sound proof system and authenticated commitments to the model weights, inputs, and proof public inputs; a malicious prover cannot fabricate a valid proof, but neither does the proof establish specification correctness or prove that the committed model, input, policy, and deployment event were the right ones. For that reason, the bundle must carry a signed manifest binding the model identifier and version, input or event identifier, policy or specification version, attestation measurement, proof public inputs, and safety-case claim identifier. Layer 3 assumes an uncorrupted hardware root of trust and a measured boot chain; physical attackers with extended access to the deployed hardware sit outside the threat model, as *TEE.fail* (TEE.fail Project, 2024) makes concrete. Layer 4 protocols vary by construction: FHE tolerates a semi-honest compute party; MPC schemes choose between semi-honest and malicious adver-

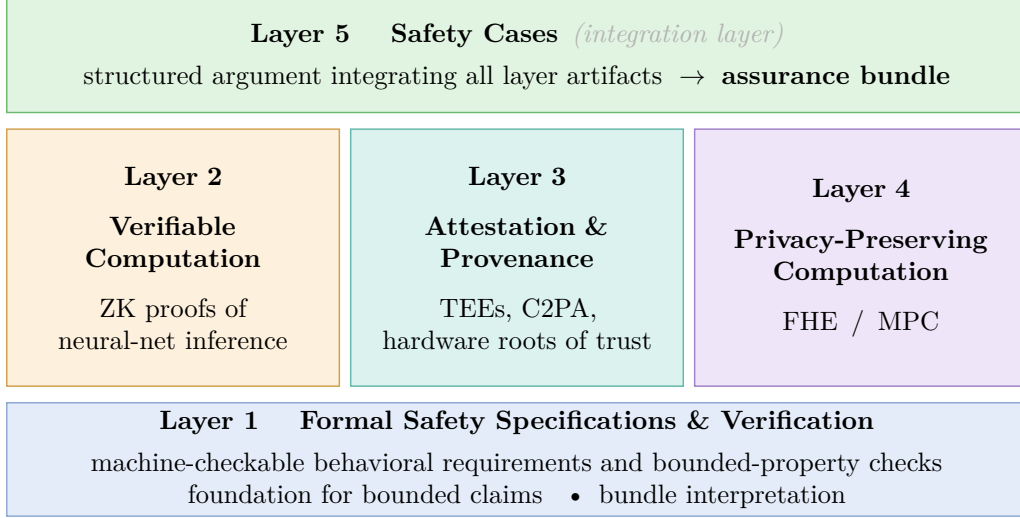


Figure 2: The five-layer composed assurance stack. Layer 1 provides the formal specifications required for bounded-property claims and for interpreting the bundle. Layers 2–4 each produce a distinct class of artifact, some machine-checkable and some provenance-bearing. Layer 5 integrates those artifacts into an *assurance bundle*: a structured package of artifacts and a safety-case argument for specified bounded claims under explicit assumptions.

sary models at a performance cost. Layer 1 specifications and Layer 5 safety cases assume a cooperative author whose remaining judgment calls are subject to independent expert review. An assurance bundle is therefore as strong as the weakest adversary assumption in its construction, and the assumption set A of §3.3 is the object that must be carried, examined, and renegotiated as deployments evolve.

3.3 What the Stack Produces

When composed, these five layers produce what we call an **assurance bundle**: the artifact that makes the vocabulary of the preceding sections concrete.

Definition. An assurance bundle accompanying a specific AI deployment contains, at minimum, six elements:

- a formal specification set Φ stating the behavioral properties the deployment claims to satisfy (Layer 1);
- *property-verification artifacts*: proof certificates and formal verification results establishing that the specified model satisfies the behavioral properties in Φ (Layers 1–2);
- *runtime-binding artifacts*: attestation reports and proof-of-inference records establishing that the deployed execution used that specific model on that specific input in that measured hardware and software environment (Layers 2–3);
- *provenance, privacy, and context artifacts*: provenance manifests establishing the origin and transformation chain of inputs and model artifacts, and privacy guarantees constraining what was exposed during execution (Layers 3–4);

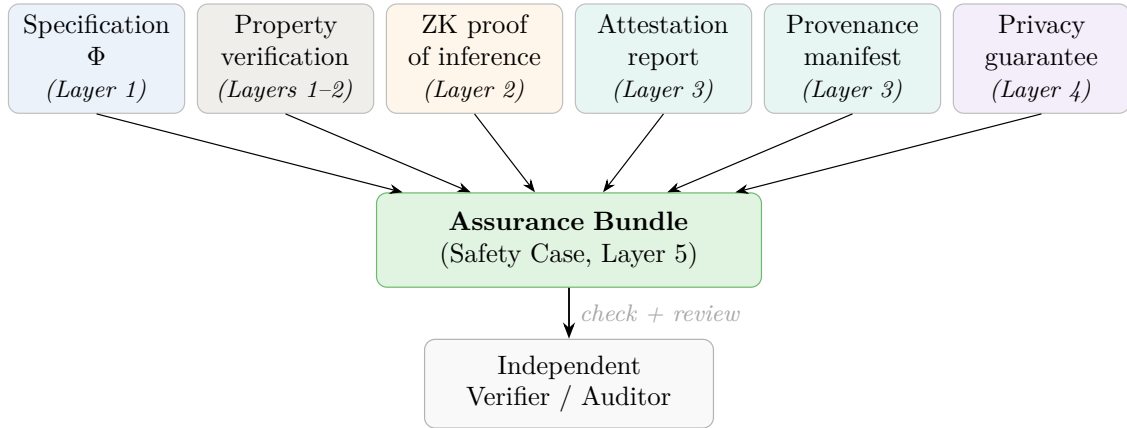


Figure 3: Composition of an assurance bundle at deployment time. Artifacts produced by Layers 1–4 are integrated by the safety case (Layer 5) into a structured bundle that a regulator or auditor can evaluate without access to the system’s internal weights or operational logs. Cryptographic artifacts (proof-of-inference records and attestation reports) and formal artifacts (property-verification certificates) are machine-checkable; the safety case is independently evaluable by qualified reviewers.

- an explicit assumption set A under which the evidence is valid (operational design domain, hardware trust model, specification faithfulness, residual risks);
- a structured safety case $\mathcal{S}$ that composes the above into a single argument a third party can evaluate without access to the provider’s internal weights or operational logs (Layer 5); the assumption set A itself—particularly the claim of specification faithfulness—remains subject to independent expert assessment and is not reducible to a mechanical check.

No element of the bundle, taken alone, supports a real-world deployment-safety claim; the bundle’s evidential weight is the composition. Bundle validity is instance-specific: the model identifier, version, input or event identifier, authority policy, operational design domain, hardware and software measurement, and update state must bind to the same deployment instance. If those identifiers do not match, the claim drops to a weaker rung of the ladder or fails entirely; if the assumption set A no longer holds, the safety case must be revised or the relevant artifacts re-generated. The signed manifest is therefore not metadata decoration; it is the object that lets a verifier determine whether the artifacts refer to the same deployment event. Figure 3 shows how artifacts from Layers 1–4 are integrated by Layer 5 into a single bundle that an independent verifier can partly machine-check and partly review.

The bundle does not prove that the system is *aligned* in any deep sense. It does not eliminate specification error. It does not address open-world behavior outside the specified operational envelope. What it does provide is something that currently does not exist: a principled, independently verifiable answer, at levels 1 and 2 of the claim ladder, to whether the claimed computation was executed and whether specified bounded properties held under stated assumptions. That is not a proof of safety in the world. It is still a great deal more than what is available today.

3.4 Scope: Objections Anticipated

Six objections to the argument above are worth stating and answering directly, because each surfaces a real line of critique the proposal must survive.

“Is this not zero-knowledge hype in safety vocabulary?” Verifiable computation is one layer of five, and the central claim is precisely that no single primitive is sufficient. A zero-knowledge proof without a formal specification establishes that some computation occurred, not that the computation was safe; attestation without privacy-preserving computation forces confidentiality trade-offs; safety cases without independently auditable evidence risk reverting to provider assertion. Machine-checkable artifacts are one important evidence class, not the only valid substrate for a safety case. The stack is a rejection of single-primitive framings, not another instance of one.

“This does not solve alignment.” Correct, and stated explicitly. The assurance claim ladder separates level-1 execution claims and level-2 bounded-property claims from level-3 claims about whether behavior is safe in the world. Alignment research, human oversight, and institutional governance remain the mechanisms by which level-3 claims are justified; the assurance stack cannot substitute for them. What it does is make the first two levels machine-checkable rather than asserted, and thereby turn level-3 into an argument about residual risk rather than an argument about all risk.

“Trusted hardware already covers most of this.” Hardware attestation is necessary and deployable today, and is included as Layer 3 for that reason. It is also insufficient on its own. Work such as *TEE.fail* (TEE.fail Project, 2024) documents concrete attacks against deployed TEE implementations; attestation addresses the physical execution environment but not the formal correctness of what was computed, nor the confidentiality requirements of multi-party deployments, nor the integration layer that turns artifacts into an audit. Layer 3 is a component of the argument, not a substitute for it.

“The proposed stack is too expensive to matter operationally.” At frontier-model scale, for general-purpose deployment, today, that is true. The near-term deployment path runs through high-value, narrow settings where proof or FHE overhead is tolerable: compliance reporting, regulated audits, safety-critical decision envelopes, cross-jurisdiction inference. The broader bet is a cost trajectory. Hardware acceleration for proving, proof-friendly architectures, and amortized verification patterns have all advanced meaningfully in recent years. Our claim is directional, not that the stack is ready for frontier-scale routine use.

“The paper conflates verification, governance, and deployment.” The layered architecture exists precisely to keep these distinct. Layer 1 is the specification-and-formal-verification layer: it states bounded behavioral predicates and checks them under explicit assumptions. Layers 2–4 produce technical artifacts of different classes. Layer 5 is the integration layer in which those artifacts become an institutional argument. The claim ladder further separates what a mechanism can prove from what an oversight process must still decide. Keeping these boundaries explicit is itself one of the contributions.

“Physical AI is carrying too much of the argument.” Physical AI is one of the two primary settings in §5; agentic delegation is the other. We mention regulated deployment and multi-provider composition more briefly because they are institutional extensions of the same assurance problem, not separate theses. The argument does not depend on any single sector: each setting makes the same point under a different failure mode.

4. Technical Primitives: State and Promise

Each layer is technically credible. None is ready to be called complete. Table 1 summarises the maturity of each primitive; the subsections below give the supporting detail. The status column reflects our assessment as of April 2026, calibrated against three criteria: whether production deployments exist in the primitive’s domain, whether research systems have achieved scale relevant to frontier-model deployment, and whether unresolved methodological gaps block routine use for general-purpose AI. The statuses are author judgments with stated grounds, not benchmarked measurements, and are intended to be read accordingly.

4.1 Formal Specifications and Verification

Formal verification of software has a long and productive history. Tools for verifying operating system kernels, cryptographic implementations, and safety-critical control systems now exist and are used in production. Extending these methods to neural networks is more recent and harder, but it is a real field, not a wishful one.

The neural network verification community traces back to Reluplex (Katz et al., 2017), which established the foundational SMT-based approach for ReLU networks. Contemporary tools such as alpha-beta-CROWN (Xu et al., 2021)—which has dominated the VNN-COMP neural network verification competition since 2021—have since scaled this substantially. These tools can prove bounded properties (robustness to perturbations within specified norms, absence of certain unsafe outputs on specified input regions) for networks of substantial size. The framework *Towards Guaranteed Safe AI* (Dalrymple et al., 2024) proposes a concrete architecture for building AI systems with formal safety guarantees, treating the problem as one of formal specification plus bounded world model plus verified policy.

What formal verification can provide is strong local guarantees over bounded input regions and specified properties, for networks of meaningful scale. The bottleneck is not proof search (verifier performance has improved substantially since 2021) but **specification**: translating the behavioral properties that matter into formal languages that verification tools can work with. This is one of the most important open problems in the field, and it attracts substantially less research effort than proof-search methods.

The specification problem has two distinct dimensions worth separating. The first is **semantic**: behavioral requirements for AI systems are typically expressed in natural language, domain norms, or organizational policy. Translating these into formal predicates that are both faithful to the intent and tractable for automated reasoning is a knowledge-engineering problem, not a proof-search problem. A medical imaging specification that says “do not recommend actions outside the clinical scope” depends on a formal definition of clinical scope that may itself require a domain ontology. The second dimension is **practical**: even where formal specifications can in principle be written, the tooling gap between what neu-

ral network verifiers accept and what practitioners without deep formal methods training can produce remains large. Program verification languages designed for working engineers rather than formal methods specialists, such as Dafny (Leino, 2010), have demonstrated that lowering the specification barrier substantially expands the population of systems that actually get verified. A comparable investment in specification tooling specifically for neural system behavioral contracts is one of the highest-leverage items on the research agenda.

A third dimension is **gaming resistance**. A learned system trained or fine-tuned against a fixed formal specification can, if sufficiently capable, satisfy the letter of the specification while violating its intent—a form of Goodhart’s Law applied to behavioral contracts. We state this observation sharply, because it sets a structural boundary on what verification alone can deliver.

Observation (Specification Gaming).

For rich open-world behavioral intents, a finite formal specification Φ is at best a proxy. A sufficiently capable learned system can in principle satisfy Φ while violating the behavioral intent behind it. This is not a limitation of any particular specification language; it is a structural consequence of the gap between formal predicates and open-world human intent.

We state this as an observation rather than a theorem: *behavioral intent* is not a formal object in the sense Φ is, so the statement is a structural claim about finite formalization rather than something that admits a proof within any particular specification language. The empirical phenomenon is documented across reinforcement learning, language models, and optimization in general (Krakovna et al., 2020), and the deeper concern—learned optimizers developing internal objectives at odds with the training signal—is analyzed under the heading of mesa-optimization and inner alignment (Hubinger et al., 2019).

We call this gap the **specification-intent gap**, and it is the boundary condition at which formal verification alone ceases to be sufficient: additional proof effort against an unchanged Φ does not by itself close the gap, because the gap lives between Φ and the intent Φ is meant to approximate. This is why alignment research is *structurally* necessary to the assurance stack rather than merely complementary to it: closing the specification-intent gap requires techniques—reward modeling, interpretability, value learning—that operate on intent rather than on predicates. The specification-intent gap is therefore one of the places where the formal methods and alignment communities must genuinely intersect, not merely coordinate. We treat it as open work and return to it in §6.

4.2 Verifiable Computation and Zero-Knowledge Proofs

Zero-knowledge proofs allow a prover to convince a verifier that a computation was performed correctly without revealing the inputs. Applied to neural network inference: given a committed model and a committed input, a provider can produce a compact proof that the observed output was generated by that model on that input, and a verifier can check the proof without re-running the computation.

Recent work has moved zkML from a theoretical curiosity to a serious research direction with genuine engineering potential. Lu et al. (2024) demonstrate verifiable inference at mean-

ingful scales, with substantial speedup over earlier baselines. Sun et al. (2023) extend the approach to training proofs—proofs that a model was trained on a specified dataset under specified conditions, a distinct and computationally more demanding problem than inference proofs, serving a different accountability purpose (auditing the training provenance rather than a specific inference event). Zhang et al. (2025) demonstrate zero-knowledge proofs of machine learning fairness properties, extending the paradigm from “did this computation occur” to “did this computation satisfy a stated property.” The fairness predicate itself, like any finite specification, inherits the specification-intent gap of §4.1: demographic parity, for instance, can be satisfied by a system whose behavior on attributes correlated with protected categories remains discriminatory. What the proof transports is the predicate chosen, not the policy the predicate was meant to stand in for.

The biggest technical risk in the assurance-stack argument lives here. ZK proofs are not ready for routine deployment on large language models or frontier vision systems. Proof generation remains computationally expensive relative to the underlying computation, and the performance gap between small-model zkML and frontier-model deployment is large. What is true is that the cost trajectory is improving, hardware acceleration for ZK proving is an active area, and the relevant near-term deployment path runs through high-value, high-stakes settings where proof overhead is acceptable (audit workflows, compliance reporting, safety-critical narrow applications) rather than general-purpose AI deployment.

Our claim about verifiable computation is directional: it is an emerging high-assurance capability, advancing quickly, and worth treating as a first-class design requirement rather than an afterthought. That is not the same as claiming it is ready.

4.3 Attestation, Roots of Trust, and Provenance

Trusted Execution Environments (TEEs) and hardware roots of trust provide a different kind of assurance: not that a computation was correct in a mathematical sense, but that it occurred on a specific, measured hardware and software stack. Remote attestation allows a remote party to verify the identity and state of a device before trusting its outputs.

Open hardware security projects such as OpenTitan (lowRISC and OpenTitan Contributors, 2024) and Caliptra (Chips Alliance, 2024) make the silicon-level trust foundation inspectable and auditable, reducing dependence on opaque proprietary implementations—though the full trust chain (integration, firmware, manufacturing process, and deployment configuration) remains a subject of ongoing security engineering. Provenance standards, particularly C2PA (Coalition for Content Provenance and Authenticity, 2024), provide a complementary mechanism at the content layer: signed metadata that establishes where content originated, what transformations it has undergone, and which systems handled it.

Attestation is practical and deployable today in meaningful contexts, with lower overhead than ZK proofs for general integrity claims. Its limits deserve equal attention. The *TEE.fail* project (TEE.fail Project, 2024) documents concrete attacks against deployed TEE implementations, underscoring that attestation should be treated as a component in a broader assurance argument rather than a definitive trust anchor. Provenance is similarly bounded: valid provenance does not establish that content is true, benign, or semantically reliable. TEEs and ZK proofs address different threat models and are most useful when combined,

each covering ground the other cannot.

4.4 Privacy-Preserving Computation

Fully homomorphic encryption (FHE) allows computation over encrypted data such that the party performing the computation learns nothing about the inputs. MPC distributes a computation across multiple parties so that no single party sees the full input. Applied to AI inference, these techniques make it possible to serve a model to multiple parties, or aggregate information from multiple sensitive sources, without centralizing or exposing the underlying data.

Ebel et al. (2023) demonstrate that FHE can be applied to deep learning at meaningful scales. Concrete ML from Zama (Zama, 2024) provides tooling for practitioners to deploy FHE-compatible models, with quantization as a key design lever. Ugurbil et al. (2025) demonstrate MPC-based approaches for private LLM inference that distribute computation across multiple servers, limiting what any single party can observe.

FHE and MPC inference remain significantly more expensive than plaintext alternatives. FHE-compatible models also typically require quantization to low bit-widths to reduce the multiplicative depth of the circuit, which can degrade accuracy compared to full-precision baselines; closing this accuracy gap is a live research problem alongside the efficiency challenge. Their strongest near-term deployment is in settings where privacy constraints are strong enough to justify the overhead: medical AI, cross-jurisdiction regulatory pipelines, and multi-party intelligence workflows. The research trajectory is encouraging; the production readiness is not yet there for general deployment.

4.5 Safety Cases

Safety cases are structured documents that make an explicit argument that a system is safe to deploy in a specified operational context, supported by enumerated evidence. They are standard in aviation (DO-178C (RTCA, Inc. and EUROCAE, 2011) mandates structured evidence of software safety objectives for flight-critical software), automotive systems (ISO 26262 (International Organization for Standardization, 2018)), and nuclear and process engineering (IEC 61508 (International Electrotechnical Commission, 2010)).

Extending safety case methodology to AI systems is nascent but active. Clymer et al. (2024) lay out one of the clearest current templates for AI safety cases, distinguishing the kinds of arguments (inability, control, trustworthiness, deference) that can justify a deployment claim and surfacing the empirical, conceptual, and mathematical evidence each requires. The basic structure (claim, argument, evidence, assumptions) maps naturally onto the artifacts the assurance stack produces. A safety case for an AI system might assert: “This medical imaging assistant, when deployed in context C , will not recommend actions outside the specified clinical decision envelope [claim]. This is supported by: (i) formal verification establishing that the specified model satisfies the decision-envelope constraints [property-verification evidence]; (ii) a proof of inference certifying that the runtime execution used that specific verified model on the specific input in the measured hardware environment [runtime-binding evidence]; and (iii) hardware attestation of the execution environment [runtime-binding evidence]. Assumptions required: sensor inputs are within the specified calibration range, and the operational environment matches the test distribution.”

Extending safety cases to systems with learned behavior, open-world inputs, and long-horizon planning introduces substantial new challenges. The framework itself is right: it forces explicitness about what is being claimed, what supports it, and what is being assumed away. That discipline is what many current deployment arguments still lack.

Having established that each primitive is technically credible but not yet mature for general-purpose use, the question becomes: where is the composed stack most urgently needed, and what does it look like in those settings?

5. Where This Matters Most

The assurance stack is not a solution for all AI deployment. It is a candidate architecture for a specific and important class: deployments where the cost of unverified behavior is high, some bounded safety-relevant claims can be meaningfully specified, and the parties involved need evidence that is evaluable independently of the system’s developers. Broader social and systemic harms—erosion of information ecosystems, labor displacement, concentration of capability—sit in governance terrain the stack supports but does not constitute; the settings below share the property that verifiable evidence about a specific deployment is load-bearing for the accountability argument.

Two settings do most of the explanatory work: agentic systems with delegated authority and physical AI. They make the integrity gap unusually stark because actions have external consequences, but they are also the settings where formalization is hardest. The stack is currently credible only where the authority surface, operational design domain, or control envelope can be compiled into finite claims that outside parties can inspect. Regulated deployment and multi-provider supply chains matter as institutional extensions of the same problem, and we treat them more briefly.

Autonomous and Agentic Systems. As AI agents are given authority to take actions in the world (executing code, managing files, sending communications, authorizing transactions), behavioral accountability becomes a structural requirement, not a nice-to-have. Who authorized this action? What was the agent’s state when it made the decision? Was the decision consistent with the delegated permission envelope? In a world where multiple AI agents interact with each other and with external systems at machine speed, these questions cannot be answered by asking the provider to review logs that only they control.

An assurance stack for agentic systems would include a formal specification of the permission envelope the agent is authorized to operate within, proof artifacts that bind specific actions to committed action traces, model and input hashes, measured runtime state, and permission policies, plus attestation of the agent’s execution environment and a provenance record of the inputs that triggered each consequential action. This does not establish the agent’s intent, rationale, or policy wisdom. It is not a complete solution to agent safety. In a bounded enterprise workflow, it is the infrastructure that can make oversight more concrete: for example, a code-deploying agent could carry evidence that a particular patch was generated by the approved model, inside an attested runtime, under a delegated permission policy that allowed changes to a staging repository but not to production credentials. That still would not prove that the patch was wise, secure, or socially acceptable; it would make

the bounded execution and authority claims independently reviewable.

Physical AI and Robotics. Physical AI introduces a qualitative change in the stakes. Actions on physical systems can cause irreversible harm. Software incidents can often preserve logs or state for remediation; physical harm cannot simply be undone by reconstructing the trace after the fact.

Physical AI also introduces the sensor trust problem: the system’s behavior is conditioned on inputs from sensors and cameras that interact with the physical world. Provenance and attestation become load-bearing here in a way they are not for pure software systems. Did the sensor data come from where it was claimed to come? Was the perception system operating within its calibrated range? Were the actuator commands verified against safety bounds before execution?

For bounded subsystems, the combination of ZK proofs for inference, hardware attestation for the execution environment, provenance standards for sensor chains, and verified control-envelope enforcement for the actuation path gives the shape of audit infrastructure this setting would need. The safety claim in physical AI typically spans the perception-to-control-to-actuation chain, not inference alone; a credible assurance argument therefore has to say which links in that chain are actually bounded and which remain outside the claim. None of this is production-ready for the full complexity of physical AI at scale. The near-term opportunity is narrower: audits of specific subsystems whose sensor, runtime, and control assumptions can be stated precisely.

Institutional Extensions: Regulated Deployment and AI Supply Chains. When AI systems are used in healthcare, financial services, law enforcement, or public administration, there are both ethical and legal reasons why behavioral claims made by system providers should be independently evaluable. The EU AI Act (European Parliament and Council of the European Union, 2024) and similar regulatory trajectories point in that direction, but the technical infrastructure to satisfy those requirements remains immature. The same problem appears in multi-provider AI pipelines, where a foundation model, adapter, retrieval corpus, and orchestration layer may come from different parties. Model cards (Mitchell et al., 2019) and datasheets for datasets (Gebru et al., 2021) are useful documentation, but they do not yet bind provenance, behavioral constraints, and integration assumptions into auditable evidence. In those settings, the assurance stack is best read as the technical substrate on which procurement, liability, and conformity-assessment regimes could rely, rather than as a separate primary use case.

This does not mean that technical evidence decides governance questions. Assurance artifacts can discipline claims about what happened, what was bounded, and which assumptions were in force; they cannot determine democratic legitimacy, acceptable risk, distributive fairness, or whether a high-impact deployment should proceed at all. Institutions still set admissible claims, refusal conditions, and thresholds for action. The stack supplies evidence for those judgments; it does not replace them.

6. Open Problems

A vision paper earns its keep not through its claims but through the quality of the research problems it generates. We identify eight.

Specification languages for AI safety properties. The hardest problem in formal verification of AI systems is not proof search (even as AI-assisted theorem proving improves rapidly (DeepSeek-AI, 2025)) but specification. We lack expressive, practical formal languages for describing the behavioral properties of AI systems that operate in open-world environments, interact with humans, and exhibit emergent behaviors from learned weights. Progress here requires work at the intersection of formal methods, programming language theory, and AI alignment: formal languages for policies, behavioral contracts for multi-agent systems, compositional specification methods that allow complex behavioral requirements to be assembled from verified components.

Scalable verifiable computation for neural inference. Current ZK proof systems for neural network inference are feasible for small to medium-scale models but remain prohibitively expensive for frontier-scale deployment. The research agenda involves more efficient proof systems tailored to the computational patterns of neural networks (matrix multiplications, activation functions, attention mechanisms), hardware acceleration for proving, and model architectures designed with proof cost as a first-class design criterion. The hypothesis that modular, world-model-style architectures (LeCun, 2022) may be more amenable to component-wise verification than monolithic transformer architectures is worth serious investigation. If verified computation is a required property for certain deployments, the implications for architecture design are significant and largely unexplored.

Compositional assurance. Each layer of the assurance stack provides a local guarantee under specific assumptions. The hard problem is composition: how do we assemble local guarantees into a coherent system-level safety argument, especially when the components come from different providers, operate under different trust models, and are assembled at runtime? This problem has analogues in compositional security, programming language semantics, and systems design. Bringing those frameworks to bear on AI assurance (composable safety types, verifiable composition operators, structured argument templates for assembled AI systems) is one of the most important open directions in the field.

Efficient privacy-preserving AI. The overhead costs of FHE and MPC remain a significant barrier, especially for large language models and vision systems. Better FHE schemes with lower multiplicative depth costs for neural network operations, quantization-aware architectures that remain accurate under the constraints of encrypted computation, and hybrid approaches that partition computation between sensitive and non-sensitive parts are all active areas that need more investment.

Safety cases for systems with learned behavior. Extending safety case methodology from traditional engineering systems (which have fixed behavior) to AI systems (which exhibit emergent, distribution-dependent behavior) requires fundamental methodological work. How do we express operational design domains precisely enough to make safety case

arguments falsifiable? How do we handle distribution shift as a safety argument? How do we compose safety cases for systems made of AI subsystems? How do we update safety cases as systems are retrained or updated? These questions are not peripheral. They are load-bearing for everything above.

Institutional admissibility and evidence thresholds. Even a well-formed assurance bundle does not decide what should be done with the evidence it contains. Institutions still need rules for admissibility, refusal thresholds, appeal, fairness review, incident reporting, and conditions under which a deployment must pause or be re-certified. The open problem is not only technical integration but governance integration: which claims must be machine-checkable, which can be expert-reviewed, which residual risks are acceptable, and who has authority to decide when the bundle is insufficient.

Lifecycle management of certified AI systems. Safety cases and assurance bundles are arguments about specific system configurations: a particular model, at a particular version, deployed in a particular operational context. AI systems are not static. They are fine-tuned, updated, adapted, and retrained. Every update potentially invalidates some part of the existing safety argument, or may be sufficiently contained that the prior argument transfers. Currently, there is no methodology for distinguishing these cases in a principled way.

The research agenda here involves two components. The first is a theory of **change impact**: formal characterization of which classes of model update, such as retraining on a bounded data distribution, low-rank adaptation within a specified behavioral envelope, or quantization to a specified precision, can be certified incrementally without re-running the full verification and safety case procedure. The second is tooling: automated pipelines that, given a prior certified system and a candidate update, determine whether the existing safety case is still valid, requires targeted re-verification, or has been fully invalidated. This problem is not peripheral to deployment practice. It is one important barrier to safety case adoption in organizations that retrain frequently, and solving it is a precondition for safety cases being used at the scale and cadence that modern AI development requires.

Continuous assurance maintenance and distribution shift. Safety cases and assurance bundles are arguments about a system operating within a specified operational design domain. Distribution shift—the gradual or sudden divergence between the observed input distribution and the domain the safety case assumed—is, by this framing, a claim that the prior assurance argument is becoming stale. This motivates a candidate monitoring function for the stack: continuous monitoring of the gap between the assumed envelope and observed inputs, with explicit alerting when the gap exceeds a threshold that would require targeted re-verification. The methodological bridge between robustness certification and safety case maintenance is underexplored. Relating out-of-distribution detection, formal operational design domain specifications, and incremental safety case update is a high-value research direction that cuts across Layers 1, 2, and 5.

7. Relation to Existing Work

Rather than surveying the adjacent literature, we position the composed assurance stack against nine clusters of prior work, each of which covers part of what the stack requires and leaves another part uncovered. The point is not that these clusters are wrong; it is that none of them, alone, produces the evidence an outside party needs to check a deployment-safety claim.

Alignment research. Alignment (Amodei et al., 2016) aims to shape what AI systems do. It operates at level 3 of the claim ladder: behavior in the world. What it does not produce is machine-checkable artifacts that connect a specific execution to a specific specification; alignment without an assurance layer yields unverifiable claims of right behavior, and the assurance stack without alignment yields verifiable evidence of wrong behavior. The closest adjacent vision is *Towards Guaranteed Safe AI* (Dalrymple et al., 2024), which decomposes high-assurance AI into a world model, a safety specification, and a verifier. We read our stack as broadening that framing to include the deployment-time evidence infrastructure (attestation, provenance, privacy-preserving computation, and structured safety cases) in which any world-model-plus-verifier argument must eventually live.

Mechanistic interpretability. Interpretability research aims to understand what AI systems are doing internally. It operates between levels 2 and 3 of the claim ladder: it can provide evidence about why a particular decision was reached, which informs level-2 specifications and level-3 oversight. What current interpretability methods do not produce is machine-checkable behavioral certificates in the sense we require: an interpretability analysis can explain what happened, but cannot itself constitute proof that a specified policy was satisfied. The relationship is complementary: interpretability findings can sharpen the formal specifications that Layer 1 requires, and a formal specification can focus interpretability investigations on the behavioral properties that matter for the safety argument. Neither is a substitute for the other.

Formal verification in software engineering. Software verification has decades of mature tooling for code with explicit semantics. The seL4 microkernel verification (Klein et al., 2009) demonstrated that machine-checked functional-correctness proofs are feasible for non-trivial, real-world systems; program-verification languages such as Dafny (Leino, 2010) have shown how to lower the specification barrier. The lessons of that tradition (modularity, abstraction, compositional reasoning) transfer directly. What does not transfer is the explicit-semantics assumption: AI systems have learned behavior and opaque internals, and the specification gap is therefore harder rather than easier. The stack treats software verification as a precedent, not an off-the-shelf solution.

Safety cases in engineering. Safety case methodology (International Electrotechnical Commission, 2010; International Organization for Standardization, 2018; RTCA, Inc. and EUROCAE, 2011), with Kelly and Weaver’s Goal Structuring Notation (Kelly and Weaver, 2004) as its canonical representation language, provides the integration framework we build on. Clymer et al. (Clymer et al., 2024) extend that framework to AI with a concrete claim taxonomy (inability, control, trustworthiness, deference). The cluster’s strength is its discipline: claims, supported by evidence, under explicit assumptions. Its gap is that

standard safety case practice presumes systems with fixed behavior and explicit design documentation; extending it to learned, open-world, re-trained systems is precisely the methodological problem Layer 5 must solve. We treat safety cases as the integration layer for the rest of the stack and treat that extension as open work.

zkML and verifiable computation. The zkML community has produced the most directly relevant technical work on verifiable AI inference, building on general-purpose zk-SNARK and zk-STARK machinery (Ben-Sasson et al., 2018; Groth, 2016) and more recently on application-specific systems for neural inference (Lu et al., 2024; Sun et al., 2023; Zhang et al., 2025). We regard this work as essential to Layer 2 but not as a complete assurance solution, and we argue that its research priorities (proof-friendly activations, hardware acceleration, composable proof systems) should be shaped by the broader architecture we describe rather than by the proof-system literature alone. Without Layers 1 and 5, a zkML proof establishes that some computation happened; it does not establish that the computation should have been trusted.

Hardware security and provenance. Hardware roots of trust, TEEs, and remote attestation provide Layer 3. Open hardware projects such as OpenTitan (lowRISC and OpenTitan Contributors, 2024) and Caliptra (Chips Alliance, 2024) make the silicon trust foundation inspectable in ways that proprietary implementations cannot match; C2PA (Coalition for Content Provenance and Authenticity, 2024) extends the provenance vocabulary to content. The cluster is the most deployment-ready in the stack. Its gap is that attestation is bounded by its threat model: work such as *TEE.fail* (TEE.fail Project, 2024) documents concrete attacks against deployed TEEs, and attestation establishes *where* something ran, not *whether what ran was correct*. Layer 3 is necessary; it is not a foundation for the full argument.

Adversarial robustness and distribution shift. Robustness research is directly relevant to Layer 1: the formal specifications verifiers check are, in many cases, robustness properties. The connection runs deeper than vocabulary. Distribution shift is, in effect, a claim that the operational design domain assumed by the safety case no longer holds. This reframes the robustness problem as a safety case problem: not “is the model robust?” but “does the deployment context remain within the domain the safety case addresses?” This reframing motivates the continuous assurance maintenance open problem described in §6: how to detect when prior assurance arguments are becoming stale and trigger targeted re-verification. Robustness certification and safety case maintenance are two faces of the same problem, and the methodological bridge between them is underexplored and consequential.

Risk taxonomies and evaluation frameworks. The broader AI safety field has named the harms a deployed system can cause (Slattery et al., 2024) and has sketched what behavioral evaluations for extreme-risk capabilities should look like (Shevlane et al., 2023). Taxonomies tell us *which* properties we should care about; evaluations tell us *how* to test for them under sampled conditions; the assurance stack is a proposal for *how to carry the resulting evidence* from the system to an outside party in a form that does not collapse un-

der scrutiny. The three are complements, not competitors, and none of them is individually sufficient.

Accountability, audit, and documentation. Work on accountable algorithms (Kroll et al., 2017), end-to-end audit (Raji et al., 2020), model cards (Mitchell et al., 2019), datasheets (Gebru et al., 2021), and conformity regimes such as the EU AI Act (European Parliament and Council of the European Union, 2024) names many of the institutional practices that make AI systems answerable. Those practices establish who must document, disclose, audit, or refuse a deployment. Their limit, for the question in this paper, is that documentation and process artifacts often remain separable from the specific computation, runtime environment, and deployment event being assessed. The assurance stack is meant to supply that binding layer, while leaving institutions responsible for deciding what evidence is admissible and what risks are acceptable.

The closest adjacent baselines therefore separate cleanly. *Towards Guaranteed Safe AI* (Dalrymple et al., 2024) provides the nearest assurance-first architecture, but centers the world-model/verifier relation rather than the deployment-time evidence pipeline. AI safety case work (Clymer et al., 2024) provides the clearest methodology for structured argument, but leaves open which technical artifacts populate that argument in practice. Proof-carrying code and zkML (Lu et al., 2024; Necula, 1997) show how producers can carry compact execution evidence, but do not by themselves address specification, runtime binding, privacy, or institutional integration. Our contribution is the deployment-time evidence architecture that assembles these lines into one stack: formalized claims, proof-producing or reviewable artifacts, runtime-binding evidence, and institutional evaluation inside a safety case.

8. Conclusion

Safety-critical engineering suggests, field by field, that accountability infrastructure is hard to improvise under pressure. Aviation, medicine, and other regulated domains developed structured evidence regimes because ordinary trust and post-hoc inspection were not enough for the decisions being made. The analogy to AI should be used carefully: AI is not aircraft, medicine, or nuclear engineering. But the lesson transfers at the level of method. Evidence infrastructure is easier to build before deployment practices harden than after.

AI is now entering domains where the same transition is becoming relevant. The empirical and behavior-shaping methods developed over the past decade are essential. They are not enough.

What is missing is an assurance layer: the technical infrastructure for producing, transmitting, and evaluating machine-checkable and independently auditable evidence about what AI systems do. That infrastructure does not yet exist in mature form. The primitives—formal verification, verifiable computation, attestation, privacy-preserving computation—exist in fragmented, partial, and often expensive form. The composition of those primitives into a coherent safety case methodology remains largely unsolved. But the work is tractable. Each layer has known problem structure. Each open problem identified above is specific enough to attract focused research effort. What is missing is integration, methodology, institutional uptake, and priority.

Closing this gap should be one of the central research directions for the assurance, integrity, and accountability slice of AI safety. Not because mathematics can substitute for judgment, or proofs can substitute for alignment, or certificates can substitute for governance, but because without machine-checkable evidence the architecture of AI deployment defaults to opacity, and accountability rests on trust in the party being held to account. Assurance by design should be treated as a first-class research agenda for high-assurance deployment contexts, complementing alignment and interpretability rather than competing with them.

The object to build is concrete: a deployment that carries inspectable evidence with it. Not a promise that the system is safe in every sense, and not a proof that governance has made the right decision, but an assurance bundle that lets an outside party examine the model, event, runtime, assumptions, and residual risks tied to a particular claim. The technical infrastructure for that kind of accountability exists in outline. Making it real, scalable, and compositional is the work.

A. Worked Example: Assurance Bundle for Diabetic Retinopathy Screening

To make the assurance bundle of §3.3 concrete, we instantiate its six elements for a specific deployment: a hypothetical AI-assisted **diabetic retinopathy screening system** operating in a primary care clinical setting. The example is fully hypothetical; it is inspired by the kind of narrow autonomous diagnostic workflow that is more tractable than open-world agency, not by a claim about any specific cleared product. We choose it because it sits in the regime where the assurance stack is most tractable: the operational envelope is bounded (retinal images from a calibrated fundus camera), the output space is binary (refer / do not refer), the workflow is regulated, and a licensed clinician reviews every positive referral before irreversible action is taken. It is the narrow end of the deployment spectrum by design.

This example is not meant to replace the agentic and physical-AI settings that motivate the paper. It is a reference case in which the same architecture can be written without pretending that open-world agency has already been solved: bounded input provenance, bounded model behavior, runtime binding, and a safety case that states where the evidence stops. It is a narrow level-2 reference case, not evidence that open-world agency is already within reach.

Element 1: Formal specification set Φ . The model shall output binary decisions (*refer* / *do not refer*) only. Let $\mathcal{D}$ denote the calibrated operational design domain and let $\mathcal{D}_v \subseteq \mathcal{D}$ denote the finite, auditable region set actually covered by the verification artifact. The model shall satisfy a robustness property ρ_ϵ on $\mathcal{D}_v$: for all input images $x \in \mathcal{D}_v$, any x' with $\|x - x'\|_\infty \leq \epsilon$ yields the same output. The decision threshold τ is fixed and audited. The model shall not produce outputs for images outside the calibrated sensor range; images inside $\mathcal{D}$ but outside $\mathcal{D}_v$ remain residual risk tracked by the safety case.

Element 2: Property-verification artifacts. An illustrative alpha-beta-CROWN (Xu et al., 2021) verification certificate establishes ρ_ϵ on the production model checkpoint over

the verified region set $\mathcal{D}_v$. The certificate is stored as a machine-checkable proof artifact cryptographically linked to the checkpoint hash, so that any third party can re-check the proof against the exact weights deployed (Layer 1). The robustness predicate ρ_ϵ is a surrogate for clinical safety, not a substitute: bounding perturbation sensitivity is a necessary but insufficient property, and closing the gap between ρ_ϵ and the bounded deployment-assurance claim is the responsibility of the safety case in Element 6, not of the certificate itself.

Element 3: Runtime-binding artifacts. A TEE attestation report (e.g., Intel TDX) binds each inference event to the verified checkpoint hash and the measured hardware and firmware environment. A *proof-of-inference* record links the input image hash $\rightarrow$ model checkpoint hash $\rightarrow$ output for every clinical decision, so that post hoc an auditor can confirm which image was processed by which verified model in which attested environment (Layers 2–3).

Element 4: Provenance, privacy, and context artifacts. Each input image carries a C2PA (Coalition for Content Provenance and Authenticity, 2024) provenance manifest recording the acquiring device ID, capture timestamp, and sensor calibration record. Where the deployment requires it, inference runs under a privacy-preserving protocol—fully homomorphic encryption, or a data-residency-compliant secure multiparty computation configuration—so patient images are not exposed in cleartext to the model host (Layers 3–4). In settings where plaintext processing is legally and clinically acceptable, the bundle should instead record that decision and its justification; the privacy-preserving layer is context-dependent, not a universal baseline.

Element 5: Assumption set A . The evidence above is valid under an explicit set of assumptions: (i) sensor inputs are within the calibrated operational range and, for the verified robustness claim, within $\mathcal{D}_v$; (ii) the patient population matches the training distribution; (iii) the clinic workflow requires a licensed physician to review all positive referrals before action is taken; and (iv) the model checkpoint has not been retrained or fine-tuned since last verification. Any violation of A invalidates the bundle’s claim.

Element 6: Safety case $\mathcal{S}$. A GSN-structured (Kelly and Weaver, 2004) argument links elements (i)–(iv) to the bounded deployment-assurance claim, showing how each piece of evidence discharges a specific sub-goal. The safety case explicitly enumerates residual risks: distribution shift outside $\mathcal{D}$, adversarial physical perturbation of the imaging sensor, deployment inputs inside $\mathcal{D}$ but outside $\mathcal{D}_v$, and the specification-intent gap—the possibility, captured by the Specification Gaming observation of §4.1, that Φ formalizes the verification target correctly but does not fully capture the clinical intent it was designed to proxy (Layer 5).

This example is narrow by design. The assurance stack is most tractable when the operational envelope can be formally bounded, the output space is small, and a human review step sits between the AI output and any irreversible action. Extending this architecture to agentic systems with open-ended action spaces, or to physical AI in unstructured environments, requires resolving the open problems catalogued in §6—in particular, specification

languages for open-world behavior and lifecycle management of certified systems under continuous update.

References

- Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565*, 2016. URL <https://arxiv.org/abs/1606.06565>.
- Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, Report 2018/046, <https://eprint.iacr.org/2018/046>, 2018.
- Chips Alliance. Caliptra: An open, reusable silicon root of trust for measurement (RoT-M). <https://github.com/chipsalliance/caliptra>, 2024. Accessed April 2026.
- Joshua Clymer, Nick Gabrieli, David Krueger, and Thomas Larsen. Safety cases: How to justify the safety of advanced AI systems. *arXiv preprint arXiv:2403.10462*, 2024. URL <https://arxiv.org/abs/2403.10462>.
- Coalition for Content Provenance and Authenticity. C2PA technical specification, version 2.0. <https://c2pa.org/specifications/specifications/2.0/>, 2024. Accessed April 2026.
- David "davidad" Dalrymple, Joar Skalse, Yoshua Bengio, Stuart Russell, Max Tegmark, Sanjit Seshia, Steve Omohundro, Christian Szegedy, Ben Goldhaber, Nora Ammann, Alessandro Abate, Joe Halpern, Clark Barrett, Ding Zhao, Tan Zhi-Xuan, Jeannette Wing, and Joshua Tenenbaum. Towards guaranteed safe AI: A framework for ensuring robust and reliable AI systems, 2024. URL <https://arxiv.org/abs/2405.06624>.
- DeepSeek-AI. DeepSeek-Prover-V2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. *arXiv preprint arXiv:2504.21801*, 2025. URL <https://arxiv.org/abs/2504.21801>.
- Austin Ebel, Karthik Garimella, and Brandon Reagen. Orion: A fully homomorphic encryption framework for deep learning. *arXiv preprint arXiv:2311.03470*, 2023. URL <https://arxiv.org/abs/2311.03470>.
- European Parliament and Council of the European Union. Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (AI act). Official Journal of the European Union, L series, <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>, 2024.
- Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021. doi: 10.1145/3458723.
- Jens Groth. On the size of pairing-based non-interactive arguments. In *Advances in Cryptology – EUROCRYPT 2016*, volume 9666 of *Lecture Notes in Computer Science*, pages 305–326. Springer, 2016. doi: 10.1007/978-3-662-49896-5_11.
- Evan Hubinger, Chris van Merwijk, Vladimir Mikulik, Joar Skalse, and Scott Garrabrant.

- Risks from learned optimization in advanced machine learning systems. *arXiv preprint arXiv:1906.01820*, 2019. URL <https://arxiv.org/abs/1906.01820>.
- International Electrotechnical Commission. IEC 61508: Functional safety of electrical/electronic/programmable electronic safety-related systems. Technical report, IEC, Geneva, Switzerland, 2010. URL <https://webstore.iec.ch/publication/22273>. Second edition.
- International Organization for Standardization. ISO 26262: Road vehicles – functional safety. Technical report, ISO, Geneva, Switzerland, 2018. URL <https://www.iso.org/standard/68383.html>. Second edition.
- Guy Katz, Clark Barrett, David L. Dill, Kyle Julian, and Mykel J. Kochenderfer. Reluplex: An efficient SMT solver for verifying deep neural networks. In *Computer Aided Verification (CAV)*, 2017. URL <https://arxiv.org/abs/1702.01135>.
- Tim Kelly and Rob Weaver. The goal structuring notation – a safety argument notation. Technical report, University of York, 2004. URL <https://www-users.cs.york.ac.uk/~tpk/dsn2004.pdf>. Proceedings of the Dependable Systems and Networks Workshop on Assurance Cases.
- Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. seL4: Formal verification of an OS kernel. In *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP)*, pages 207–220. ACM, 2009. doi: 10.1145/1629575.1629596.
- Victoria Krakovna, Jonathan Uesato, Vladimir Mikulik, Matthew Rahtz, Tom Everitt, Ramana Kumar, Zac Kenton, Jan Leike, and Shane Legg. Specification gaming: The flip side of AI ingenuity. DeepMind Research Blog, <https://deepmind.google/discover/blog/specification-gaming-the-flip-side-of-ai-ingenuity/>, 2020. Accessed April 2026.
- Joshua A. Kroll, Joanna Huey, Solon Barocas, Edward W. Felten, Joel R. Reidenberg, David G. Robinson, and Harlan Yu. Accountable algorithms. *University of Pennsylvania Law Review*, 165:633–705, 2017. URL https://scholarship.law.upenn.edu/penn_law_review/vol165/iss3/3/.
- Yann LeCun. A path towards autonomous machine intelligence. Technical report, Meta AI Research, 2022. URL <https://openreview.net/pdf?id=BZ5a1r-kVsf>.
- K. Rustan M. Leino. Dafny: An automatic program verifier for functional correctness. In *Logic for Programming, Artificial Intelligence, and Reasoning (LPAR-16)*, volume 6355 of *Lecture Notes in Computer Science*, pages 348–370. Springer, 2010. doi: 10.1007/978-3-642-17511-4_20.
- lowRISC and OpenTitan Contributors. OpenTitan: Open source silicon root of trust. <https://opentitan.org/>, 2024. Accessed April 2026.
- Tao Lu, Haoyu Wang, Wenjie Qu, Zonghui Wang, Jinye He, Tianyang Tao, Wenzhi Chen, and Jiaheng Zhang. An efficient and extensible zero-knowledge proof framework for

- neural networks. Cryptology ePrint Archive, Report 2024/703, <https://eprint.iacr.org/2024/703>, 2024.
- Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT*)*, pages 220–229. ACM, 2019. doi: 10.1145/3287560.3287596.
- George C. Necula. Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 106–119. ACM, 1997. doi: 10.1145/263699.263712.
- Inioluwa Deborah Raji, Andrew Smart, Rebecca N. White, Margaret Mitchell, Timnit Gebru, Ben Hutchinson, Jamila Smith-Loud, Daniel Theron, and Parker Barnes. Closing the AI accountability gap: Defining an end-to-end framework for internal algorithmic auditing. In *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAccT)*, pages 33–44. ACM, 2020. doi: 10.1145/3351095.3372873.
- RTCA, Inc. and EUROCAE. DO-178C / ED-12C: Software considerations in airborne systems and equipment certification. Technical report, RTCA, Inc., Washington, DC, 2011. URL <https://www.rtca.org/content/standards-guidance-materials>. Aviation software safety standard.
- Stuart Russell, Daniel Dewey, and Max Tegmark. Research priorities for robust and beneficial AI. *AI Magazine*, 36(4):105–114, 2015. URL <https://arxiv.org/abs/1602.03506>.
- Toby Shevlane, Sebastian Farquhar, Ben Garfinkel, Mary Phuong, Jess Whittlestone, et al. Model evaluation for extreme risks. *arXiv preprint arXiv:2305.15324*, 2023. URL <https://arxiv.org/abs/2305.15324>.
- Peter Slattery, Alexander K. Saeri, Emily A. C. Grundy, Jess Graham, Michael Noetel, Risto Uuk, James Dao, Soroush Pour, Stephen Casper, and Neil Thompson. The AI risk repository: A comprehensive meta-review, database, and taxonomy of risks from artificial intelligence. *arXiv preprint arXiv:2408.12622*, 2024. URL <https://arxiv.org/abs/2408.12622>.
- Haochen Sun, Tonghe Bai, Jason Li, and Hongyang Zhang. zkDL: Efficient zero-knowledge proofs of deep learning training. Cryptology ePrint Archive, Report 2023/1174, <https://eprint.iacr.org/2023/1174>, 2023.
- TEE.fail Project. TEE.fail: Practical attacks against trusted execution environments. <https://tee.fail/>, 2024. Accessed April 2026.
- Mehmet Ugurbil, Dimitris Mouris, Manuel B. Santos, José Cabrero-Holgueras, Miguel de Vega, and Shubho Sengupta. Fission: Distributed privacy-preserving large language model inference. Cryptology ePrint Archive, Report 2025/653, <https://eprint.iacr.org/2025/653>, 2025.
- Kaidi Xu, Zhouxing Shi, Huan Zhang, Yihan Wang, Kai-Wei Chang, Minlie Huang, Bhavya Kailkhura, Xue Lin, and Cho-Jui Hsieh. Fast and complete: Enabling complete neural network verification with rapid and massively parallel incomplete verifiers. In *Proceedings*

of the International Conference on Learning Representations (ICLR), 2021. URL <https://openreview.net/forum?id=nVZtXBI6LNn>.

Zama. Concrete ML: A privacy-preserving machine learning library using fully homomorphic encryption, 2024. URL <https://docs.zama.ai/concrete-ml>.

Tianyu Zhang, Shen Dong, O. Deniz Kose, Yanning Shen, and Yupeng Zhang. FairZK: A scalable system to prove machine learning fairness in zero-knowledge. *arXiv preprint arXiv:2505.07997*, 2025. URL <https://arxiv.org/abs/2505.07997>.

This paper is a position paper and research agenda. It does not present new empirical results. Earlier versions of the ideas were developed as private research notes. The author thanks the growing communities working on formal verification, zkML, hardware security, and AI safety for the foundations this argument builds on.

Table 1: State of the assurance stack primitives. Status: **Practical** = deployable today in relevant contexts; **Emerging** = advancing rapidly, meaningful scale achieved; **Research** = proven in principle, significant gaps remain; **Nascent** = early methodological work.

Primitive	Evidence it yields	Key limitation	Status
Formal specs & verification	Bounded-property evidence over specified input regions; absence of certain unsafe outputs under Φ	Specification bottleneck: translating behavioral requirements to formal language	Emerging
Verifiable computation (ZK)	Committed model executed committed input to produce committed output	Proof generation too expensive for frontier-scale routine deployment	Research
Attestation & provenance	Hardware identity, execution environment, content origin and chain-of-custody	Limited threat model; not absolute against physical attackers	Practical
Privacy-preserving comp. (FHE/MPC)	Input confidentiality in multi-party or cross-jurisdiction settings	Significant overhead vs. plaintext inference; quantization constraints	Research
Safety cases	Structured argument linking evidence, assumptions, residual risks, and stated requirements	No established methodology for learned-behavior or open-world AI systems	Nascent